DBI Mitschrift

Graphen, JSON & SQL vs. NoSQL

Inhaltsverzeichnis

1. Grundbegriffe 1
   1.1. Beispiel: Städte 1
2. Gerichtete und ungerichtete Graphen 1
3. Anwendung: Ähnlichkeiten erkennen 2
   3.1. Beispiel: Nintendo Switch 2
4. Technologien 2
5. Allgemeines 4
6. JSON & dokumentorientierte Datenbanken 4
   6.1. JSON-Grundlagen 4
   6.2. Beispiel: Religionsanmeldung 4
   6.3. CAP-Theorem 5
7. CRUD: SQL vs. NoSQL 5
8. Schätzung Supermarkt 6
9. Projekt 6

1. Grundbegriffe

Entität Ein konkretes Objekt, z.B. *Jurij* ist eine Entität vom Entitätstyp *Schüler*.

Entitätstyp Die Kategorie / Klasse von Entitäten, z.B. *Schüler*.

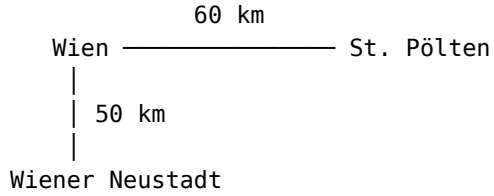
Attribut Eigenschaft einer einzelnen Entität (z.B. Name, Einwohnerzahl).

Beziehung (Kante) Verbindung zwischen zwei Entitäten. Auch Beziehungen können Werte (Attribute) haben.

In einem Graphen sind die Entitäten die **Knoten** (Nodes), die Beziehungen die **Kanten** (Edges).

Wichtig: Entitäten stehen in Beziehung zu anderen Entitäten – aber **nicht alle** zueinander.

1.1. Beispiel: Städte



- Knoten: Wien, St. Pölten, Wiener Neustadt (mit eigenen Attributen, z.B. Einwohnerzahl)
- Kanten: Straßenverbindungen mit dem Wert *Distanz* (ein Graph mit Kantenwerten heißt **gewichteter Graph**)

2. Gerichtete und ungerichtete Graphen

> **Prüfungsrelevant:** Unterschied zwischen gerichteten und ungerichteten Graphen.

Ungerichtet Die Beziehung gilt in beide Richtungen gleich.
Beispiel: Die Distanz Wien ⟶ St. Pölten ist dieselbe wie St. Pölten ⟶ Wien.

Gerichtet Die Beziehung hat eine Richtung, der Wert kann je Richtung verschieden sein.
Beispiel: „Wer eine Switch kauft, kauft auch einen Controller“ hat einen anderen Wert als „Wer einen Controller kauft, kauft auch eine Switch“.

3. Anwendung: Ähnlichkeiten erkennen

Graphen zeigen Ähnlichkeiten bzw. Zusammenhänge, z.B. bei Amazon: *Welche Produkte werden zusammen gekauft?* (Empfehlungssysteme)

3.1. Beispiel: Nintendo Switch

- Wer eine Switch kauft, kauft oft auch ein **Spiel** und einen **Controller**.
- Über zwei Schritte (Switch → Controller/Spiel → …) kommt man z.B. zu **Transportboxen**.
- Die Kanten tragen einen Faktor, z.B.: von 1000 Switch-Käufern kaufen 900 auch ein Spiel (900 / 1000 = 0,9).
- Da der Faktor von der Ausgangsseite abhängt, ist dieser Graph **gerichtet**.

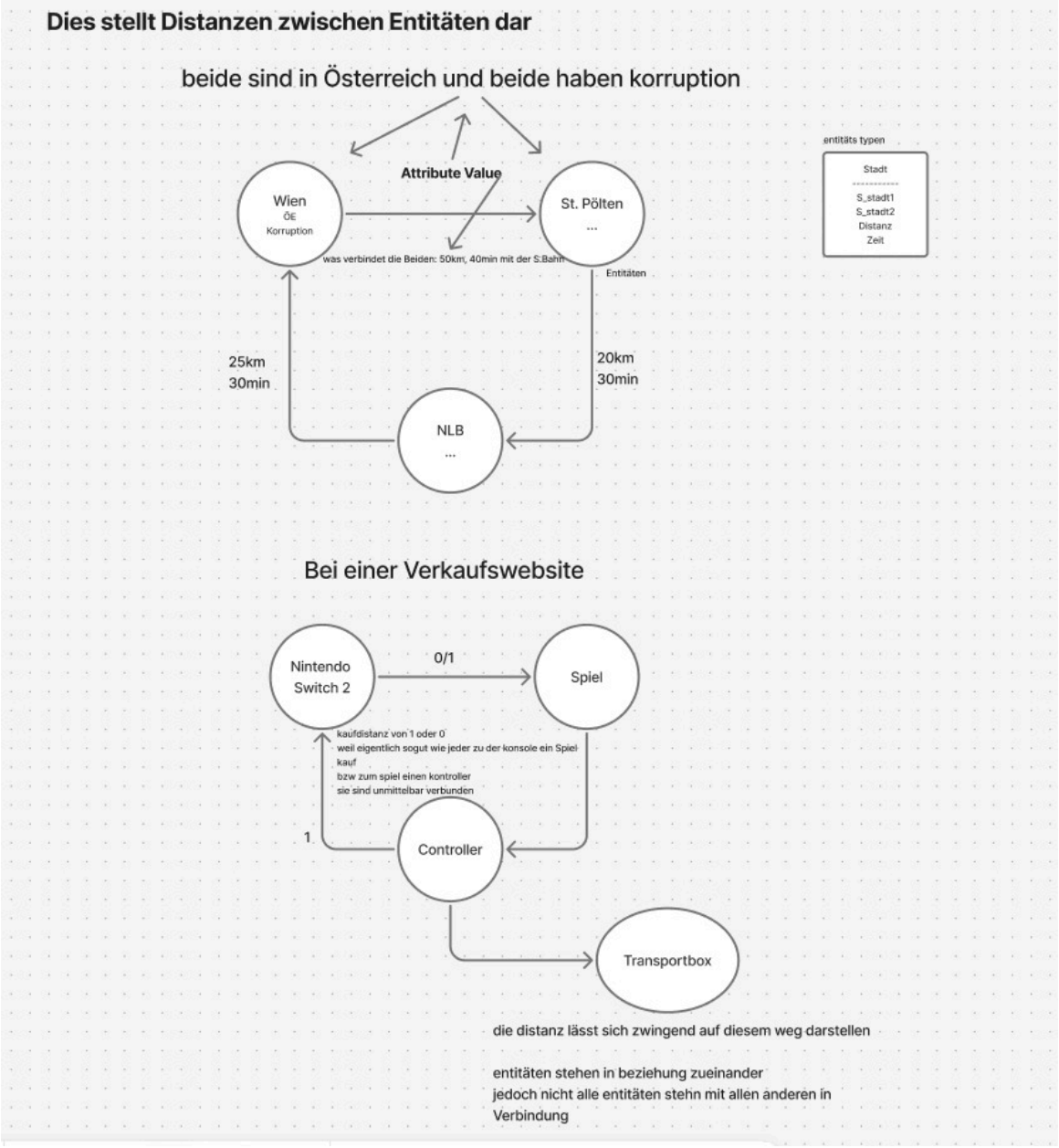


Abbildung 1: Distanzen zwischen Entitäten (Städte) und Kaufbeziehungen bei einer Verkaufswebsite

4. Technologien

- **Graphdatenbanken** (z.B. Neo4j) speichern Daten direkt als Knoten und Kanten.
- Vorteil: Beziehungen über mehrere Schritte („Freunde von Freunden“, „Kunden kauften auch …“) lassen sich schnell abfragen – in SQL bräuchte man dafür viele Joins.

Ergänzung: Neo4j verwendet die Abfragesprache **Cypher**, z.B.

```
MATCH (:Produkt {name: "Switch"})-[:GEKAUFT_MIT]->(p) RETURN p
```

5. Allgemeines

- Prüfung (Niklas) am Papier – JSON-Dokumente werden auf dem Blatt Papier geschrieben
- Themen: relationale Datenbanken und NoSQL
- Voraussetzung: grobe DB-Kenntnisse (ER-Modell, SQL)

6. JSON & dokumentorientierte Datenbanken

6.1. JSON-Grundlagen

JSON (*JavaScript Object Notation*) speichert Daten als Schlüssel-Wert-Paare.

- `{ }` → **Objekt** (Schlüssel-Wert-Paare, z.B. `{ "name": "Neil" }`)
- `[ ]` → **Array** (Liste von Werten)
- `[{ }]` → Array von eingebetteten Objekten (z.B. mehrere Schuljahre eines Schülers)
- Datentypen: String, Zahl, Boolean, `null`, Objekt, Array

Ergänzung: In MongoDB entspricht ein JSON-Dokument einer Zeile, eine **Collection** einer Tabelle:

Relational (SQL)	MongoDB (NoSQL)
Tabelle	Collection
Zeile / Datensatz	Dokument
Spalte	Feld
Primärschlüssel	`_id`
Join	Einbetten (bzw. `$lookup`)

6.2. Beispiel: Religionsanmeldung

Simples DB-Model fürs Anmelden zum Religionsunterricht:

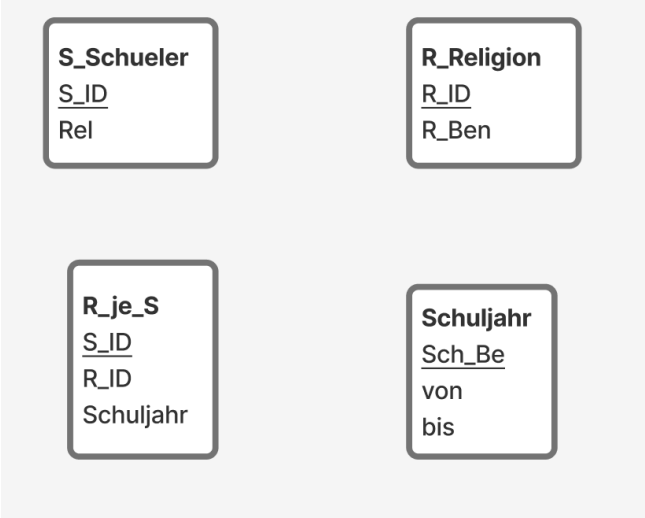


Abbildung 2: DB-Model Religionsanmeldung

6.2.1. Variante 1: Pro Schüler ein Dokument

```
db.RelJeSchueler.insertOne({
  name: "Gundi",
  RelBezeichnung: "…",
  Schuljahre: [
    { Schuljahr: "26/27", gewaehlterUnterricht: "…", Klasse: "…" }
```

```
  ]
})
```

6.2.2. Variante 2: Pro Religion ein Dokument

Collection `SchuelerJeReligion` (die Religion dient als `_id`):

```
db.SchuelerJeReligion.insertOne({
  _id: "…",            // Religion
  Schuljahr: "26/27",
  Schueler: [{
    id: …,
    schuelerfirstname: "…",
    schuelerlastname: "…",
    note: …            // eher ungünstig, siehe unten
  }],
  Lehrer: [{
    id: …,
    lehrername: "…"
  }]
})
```

Für jede Religion wird eine Liste an Schülern und eine Liste von Lehrern gespeichert, welche dieses Fach unterrichten. Für die **Historisierung** wird zusätzlich das Schuljahr gespeichert.

> **Ergänzung:** Warum ist `note` im Schüler-Array ungünstig? Das Dokument wächst mit jedem Schüler; um eine einzelne Note zu ändern, muss im Array gesucht werden. Daten, die sich oft einzeln ändern, besser in eigenen Dokumenten speichern.

> **Prüfungsrelevant:** Ein DB-Model (ER-Modell) in JSON-Dokumente umwandeln können. Entscheidend ist, **wovon man ausgeht** (Schüler oder Religion) – danach richtet sich, was eingebettet wird. Zusammengehörende Daten, die gemeinsam gelesen werden, werden **eingebettet** statt per Join verknüpft.

6.3. CAP-Theorem

Ein verteiltes System kann von den folgenden drei Eigenschaften nur **zwei gleichzeitig** garantieren:

C – Consistency Alle Knoten sehen zur selben Zeit dieselben Daten.
A – Availability Jede Anfrage bekommt eine Antwort.
P – Partition Tolerance Das System funktioniert weiter, auch wenn die Verbindung zwischen Knoten ausfällt.

> **Ergänzung:** Da Netzwerkausfälle in verteilten Systemen nie ganz auszuschließen sind, muss man in der Praxis meist zwischen **CP** (z.B. MongoDB) und **AP** (z.B. Cassandra) wählen. Klassische relationale DBs auf einem Server gelten als **CA**.

7. CRUD: SQL vs. NoSQL

CRUD – **C*reate,** R*ead, **U*pdate,** D*elete

	SQL	**NoSQL**
C	Langsam (Constraints, Indizes, Fremdschlüssel prüfen)	Sehr schnell (kein festes Schema)

	SQL	NoSQL
R	Schnell	Langsamer – bei kleinen Datenmengen aber sogar schneller als SQL, weil der Join wegfällt
U	Mit Filter schnell Ohne Filter mittelschnell	Mit Filter langsam Ohne Filter eher schnell
D	Mit Filter langsam Ohne Filter langsam	Mit Filter mittel Ohne Filter schnell – sehr schnell

Wir löschen alle Datensätze – warum braucht das bei SQL einen zweiten Prozess?
→ Es müssen **Abhängigkeiten** (Fremdschlüssel, referentielle Integrität) geprüft werden.

Eine relationale DB hat eine **DDL** (*Data Definition Language*: `CREATE`, `ALTER`, `DROP`) – das Schema wird vorher festgelegt. NoSQL-Datenbanken sind meist **schemalos**: Dokumente einer Collection können unterschiedlich aufgebaut sein.

8. Schätzung Supermarkt

Wie viele Datensätze und Operationen gibt es?

Entität	Datensätze	Änderungen
Filialen	~1.000	~10 neue pro Jahr
Produkte	~40.000	~1.000 neue pro Jahr, bis zu 2 Mio. Operationen pro Jahr (z.B. Preisänderungen)
Rechnungen	~10.000.000 pro Tag (Anzahl Einkäufe)	laufend neue
Positionen	min. 5 pro Rechnung → ~50.000.000 pro Tag	laufend neue

- Filialen und Produkte ändern sich selten, werden aber ständig gelesen → **leselastig** → relationale DB gut geeignet.
- Rechnungen und Positionen entstehen in riesiger Zahl → **create-lastig** → hier spielt NoSQL seine Stärke beim Einfügen aus.
- Für Auswertungen (Umsatz, Statistiken) werden die Positionen anschließend viel gelesen.

Prüfungsrelevant:
- Beurteilen, ob ein Prozess **leselastig** oder **create-lastig** ist, und begründet empfehlen, welche DB man verwendet.
- Die Mengen (Datensätze, Operationen) aus dem ER-Modell bzw. der Angabe abschätzen.

9. Projekt

- Zweiergruppen
- Theoretische Grundlage vorausgesetzt (wird praktisch nicht unterrichtet)
- 4 Tabellen (1 Lookup-Tabelle und 2–3 Kreuztabellen)
- Neben der relationalen Schicht ein zweiter Ast, der auf eine NoSQL-Datenbank (MongoDB) geht
- Mit 3 verschiedenen Datenmengen (klein, mittel, groß – groß soll mindestens 1 Minute dauern) die Laufzeiten der CRUD-Operationen beider Systeme vergleichen

3. Dezember 2026: Abgabe und Abgabegespräche (bei späterer Abgabe gibt es Punkteabzug)